

Assessing Geometric Security of AES Neural Realizations: Linear-Time Key Recovery via Neural Leakage^{*}

Kwangjo Kim

KAIST/IRCS
kkj@kaist.ac.kr

Abstract. We investigate the security of AES-128/192/256 when implemented as ReLU-based neural networks via the natural sum-of-corners construction. Although these implementations are bit-exact on Boolean inputs, they extend AES into a continuous piecewise-linear function over $\mathbb{R}^{128}$. We show that under real-valued oracle access, such neural realizations admit *deterministic* linear-time master-key recovery. The attack exploits a geometric property of the natural XOR (AddRoundKey) layer: for corner parameter $c < 1$, ReLU activations partition the input space into key-dependent linear regions. Using symmetric perturbations, exactly one key hypothesis preserves linear-region membership, enabling bitwise recovery through simple output-equality tests. We formalize this phenomenon via a local separability lemma and obtain attack complexity $O(128R)$ neural queries for R rounds. Experiments over 1,000 independent random keys for AES-128, AES-192, and AES-256 achieve 100% recovery success. The vulnerability is independent of key size and round count; it stems solely from exposing a key-dependent XOR layer inside a continuous neural architecture. Our results reveal a fundamental gap between Boolean security and geometric security in neural implementations of cryptographic primitives.

Keywords: Neural Cryptanalysis, ReLU Networks, Activation-Boundary Leakage, Synthetic Oracle Model, AES Key Recovery

1 Introduction

Block ciphers are one of the most fundamental cryptographic primitives, designed to provide confidentiality through fast and deterministic computation. From a theoretical perspective, an ideal block cipher is modeled as a random permutation, while practical constructions aim to approximate this ideal behavior using efficiently computable functions. The security of block ciphers relies on the assumption that their internal structure is indistinguishable from a random function or permutation to any efficient adversary, despite being fully specified and publicly known. The history of block cipher cryptanalysis reflects continuous progress in understanding and exploiting structural weaknesses. Beginning with differential cryptanalysis [3] and linear cryptanalysis [20] against DES (Data Encryption Standard) [23], cryptanalytic techniques have evolved into a wide range of sophisticated variants, including side-channel attacks [17], higher-order attacks [16, 18, 27] and algebraic attacks [5]. These advances have consistently reshaped cipher design principles, reinforcing the importance of rigorous security analysis and conservative design margins.

In parallel, AI (Artificial Intelligence) has undergone its own transformative evolution. Modern AI systems, particularly deep neural networks, construct complex functions by learning from large datasets rather than being explicitly designed by humans. Landmark events such as the AlphaGo [8] match against Professional Go player, Sedol Lee demonstrated the power of learning-based systems to surpass human-crafted heuristics

^{*} In accordance with responsible disclosure practices, we have proactively briefed the appropriate standardization organizations and security authorities on these results to collaborate on developing robust countermeasures.

in highly structured domains. At its core, AI represents a paradigm in which functions are implicitly generated through optimization and training, guided by data rather than explicit mathematical construction.

This contrast reveals an intriguing conceptual symmetry between cryptography and AI. In cryptography, humans deliberately design functions whose internal structure is difficult to analyze, aiming to resist all feasible forms of learning or inference by an adversary. In AI, by contrast, the goal is to learn an unknown function as accurately as possible from observed input–output behavior. Both fields revolve around the construction, approximation, and analysis of functions, but with fundamentally opposing objectives.

The recent application of neural networks to cryptanalysis raises important questions at the intersection of these two domains. Can learning-based models approximate or exploit the behavior of cryptographic functions? If so, under what assumptions and limitations? Understanding these questions requires revisiting classical cryptographic models in light of modern learning paradigms, and reassessing whether long-standing security intuitions remain valid when attackers are equipped with powerful learning tools. The widespread adoption of DNNs (Deep Neural Networks) raises fundamental questions about how to endow them with cryptographic functionalities such as decryption, authorization, or secure watermarking. Cryptographic primitives are traditionally designed for digital computers operating on discrete Boolean gates, whereas DNNs are analog computational models based on linear transformations and ReLU activations over real-valued inputs. This mismatch between discrete and continuous computation introduces new security challenges, particularly when adversaries can query a cryptographic DNN with nonstandard, real-valued inputs.

The seminal work [10] establishes foundational definitions of correctness and security for cryptographic primitives implemented as ReLU-based DNNs, and demonstrate that natural DNN implementations of block ciphers are inherently insecure in this setting. Specifically, they show that full-round AES (Advanced Encryption Standard) [26] with 128 bit master key implemented as a DNN can be broken in linear time using real-valued queries, successfully recovering 1,000 randomly chosen keys with 100% success. To address this vulnerability, they propose a new, provably secure method for implementing arbitrary cryptographic functionalities as standard ReLU-based DNNs, achieving correctness and security against such attacks with only constant additional layers and a linear increase in neurons, making the approach fully practical.

In this work, we experimentally investigate DNN-based key recovery attacks against the NIST standard block cipher AES-128/192/256. Unlike classical statistical cryptanalysis, our approach leverages synthetic oracle constructions derived from neural representations of the AddRoundKey (XOR) layer.

We provide:

- A full neural correctness verification of AES-128/192/256
- A DNN-based round-key recovery framework
- Two pair-generation strategies ($\text{GenPairs}_{\text{change}}$ and $\text{GenPairs}_{\text{sym}}$)
- Extensive 1,000-trial experimental validation for each key size

Our results show that $\text{GenPairs}_{\text{sym}}$ achieves consistent full master-key recovery across all AES variants, while $\text{GenPairs}_{\text{change}}$ exhibits structural limitations depending on key schedule propagation, but succeeded to recover the random master key of AES-128 only.

Organization of the Paper. The remainder of this paper is organized as follows. **Section 2** introduces the necessary preliminaries on Boolean functions, ReLU-based deep neural

networks, and the formal threat model for DNN-based cryptographic implementations. **Section 3** presents the natural sum-of-corners construction for realizing Boolean functions as ReLU networks, including the XOR case and its geometric interpretation. **Section 4** describes the full natural neural implementation of AES-128/192/256, covering AddRound-Key, SubBytes, ShiftRows, and MixColumns. **Section 5** develops the core key-recovery framework, including the pair-generation mechanisms and the local separability lemma underlying the attack. **Section 6** provides the complete experimental validation for AES-128, including Monte-Carlo evaluation and parameter sweeps. **Section 7** analyzes round-key recoverability in AES-192 under both standard black-box and synthetic intermediate-state models. **Section 8** extends the synthetic-oracle analysis to AES-256 and evaluates computational cost. **Section 9** discusses related work at the intersection of neural networks and cryptography. Finally, **Section 10** concludes with implications for neural implementations of cryptographic primitives and directions for future research.

This study aims to provide a rigorous security benchmark for neural-integrated cryptographic systems, ensuring that future AI-driven security architectures are resilient against such geometric exploitation.

2 Preliminaries

Let $\mathbb{F}_2$ denote the binary field with two elements. The set $\mathbb{F}_2^n$ is the set of all binary vectors of length n , viewed as an $\mathbb{F}_2$ -vector space.

A Boolean function is a function

$$f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2.$$

The set of all Boolean functions from $\mathbb{F}_2^n$ to $\mathbb{F}_2$ is denoted by $\mathcal{B}_n$.

We assume implicitly that an ordering of $\mathbb{F}_2^n$ is fixed, so that

$$\mathbb{F}_2^n = \{x_1, x_2, \dots, x_{2^n}\}.$$

A Boolean function f can be specified by its truth table (or evaluation vector), which gives the value of f at all x_i . Once the order on $\mathbb{F}_2^n$ is chosen, the truth table of f uniquely determines f .

A Boolean function $f \in \mathcal{B}_n$ can also be expressed uniquely as a square-free polynomial over

$$\mathbb{F}_2[X] = \mathbb{F}_2[x_1, \dots, x_n].$$

More precisely,

$$f(x_1, \dots, x_n) = \sum_{(v_1, \dots, v_n) \in \mathbb{F}_2^n} b(v_1, \dots, v_n) x_1^{v_1} \cdots x_n^{v_n},$$

where $b(v_1, \dots, v_n) \in \mathbb{F}_2$. This representation is called the *Algebraic Normal Form (ANF)*.

There exists a simple divide-and-conquer butterfly algorithm [4] to compute the ANF from the truth table (or vice versa) of a Boolean function. This algorithm requires $O(n2^n)$ bit additions and $O(2^n)$ bits of storage. It is known as the *fast Möbius transform*.

2.1 Basic DNN Definitions

Definition 1 (ReLU Activation Function). *The ReLU (Rectified Linear Unit) activation function maps negative values to zero and is defined as*

$$\text{ReLU}(x) = \max\{0, x\}.$$

Definition 2 (Neuron). A neuron is a function η determined by a weight (column) vector w , a bias b , and an activation function f , which computes

$$\eta(x) = f(x \cdot w + b).$$

Definition 3 (Deep Neural Network (DNN)). A Deep Neural Network (DNN) is a function

$$f : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_{r+1}}$$

constructed as a composition of neurons arranged in layers. Layer l is characterized by a weight matrix $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$, a bias vector $b_l \in \mathbb{R}^{d_l}$, and activation functions σ_l , where neuron j in layer l has weight vector $W_{l,j}$ (the j -th row of W_l), bias $b_{l,j}$, and activation function $\sigma_{l,j}$.

Throughout this paper, we focus on ReLU-based neural networks, which consist only of linear mappings and ReLU activation functions. We sometimes loosely use the term *DNN* to refer to such ReLU-based neural networks.

2.2 Our Threat Model

Let D be a DNN-based implementation of some binary keyed cryptographic primitive B (e.g., AES encryption, HMAC, RSA signing, etc.). The binary implementation B has a binary key space $\mathcal{K} \subseteq \{0, 1\}^\lambda$, a binary input space $\mathcal{P}_{\text{Bin}} \subset \{0, 1\}^*$, and a binary output space $\mathcal{C}_{\text{Bin}} \subset \{0, 1\}^*$, such that

$$B : \mathcal{K} \times \mathcal{P}_{\text{Bin}} \rightarrow \mathcal{C}_{\text{Bin}}.$$

The DNN-based implementation of the same cryptographic primitive D has (the same) key space $\mathcal{K} \subseteq \{0, 1\}^\lambda$, an infinite-precision real-valued input space $\mathcal{P}_{\mathbb{R}} \subset \mathbb{R}^*$, and an output space $\mathcal{C}_{\mathbb{R}} \subset \mathbb{R}^*$, such that

$$D : \mathcal{K} \times \mathcal{P}_{\mathbb{R}} \rightarrow \mathcal{C}_{\mathbb{R}}.$$

For the sake of brevity, we will omit the key from our notation unless it is explicitly required. Throughout this paper, we assume that our DNN-based implementation D is *correct*, i.e., it agrees with the binary implementation B on all binary inputs.

Definition 4 (Correctness of DNN-based Implementation). For any DNN-based implementation D of a binary cryptographic primitive B , we say that D is correct if

$$\forall k \in \mathcal{K} \ \wedge \ \forall p \in \mathcal{P}_{\text{Bin}} : \quad B_k(p) = D_k(p).$$

Our threat model assumes an adaptive attacker with oracle access to D . The attacker can query the output value of D on any chosen input, where *each input “bit” is allowed to be an arbitrary real number*. The attacker is adaptive, meaning that each query may depend on the outputs of previous queries.

3 Natural Implementation of Boolean Functions

In this section, we describe the simplest way to implement any Boolean function over a small number of bits as a ReLU-based neural network, which we refer to throughout the rest of the paper as its *natural implementation*. We first illustrate the technique using the example of the XOR of two bits, before describing the arbitrary Boolean functions.

3.1 Natural Implementation of XORing two inputs

The XOR function of two inputs x_1, x_2 can be implemented using a simple one-layer DNN with two neurons, which yields the following intermediate values C_1 and C_2 :

$$C_1 = \text{ReLU}(x_2 - x_1), \quad C_2 = \text{ReLU}(x_1 - x_2).$$

The output

$$\text{NN}_{\text{XOR}} = C_1 + C_2 \tag{1}$$

corresponds to the absolute value of the difference between the two inputs, and thus measures their real-valued distance in Euclidean domain. For any combination of $x_1, x_2 \in \{0, 1\}$, this output produces the desired XOR value, while also providing a continuous extension of XOR to $\mathbb{R}^2$ compared to the binary addition over $\mathbb{F}_2^2$. In other word, this neural operator preserves bit-exact correctness over $\{0, 1\}^2$ while admitting a piecewise-linear extension over $[0, 1]^2$, which is essential for subsequent slope-based analysis.

3.2 Natural Implementation of Arbitrary Boolean Functions

We now describe a neural realization of an arbitrary Boolean function $f : \{0, 1\}^k \rightarrow \{0, 1\}$ based on a geometric decomposition of the Boolean hypercube.

Rather than viewing f as a truth table, we interpret it as the indicator function of a subset of vertices of the hypercube. Let

$$\mathcal{C} := \{b \in \{0, 1\}^k \mid f(b) = 1\}$$

denote the support of f . Each element of $\mathcal{C}$ will be associated with one neuron in a single hidden layer. Thus, the number of hidden neurons equals $|\mathcal{C}| \leq 2^k$, which is practical whenever k is a small constant (e.g., $k = 8$ in the case of AES S-boxes).

For each vertex $b = (b_1, \dots, b_k) \in \mathcal{C}$, we construct an affine function

$$\ell_b(x) = \langle w_b, x \rangle + \beta_b$$

whose positive region contains only the point b among all vertices of $\{0, 1\}^k$. Geometrically, the associated hyperplane

$$\ell_b(x) = 0$$

is oriented such that b lies strictly in the positive half-space, while every other Boolean vertex lies in the non-positive half-space.

After applying a ReLU activation,

$$\phi_b(x) = \text{ReLU}(\ell_b(x)),$$

the neuron outputs a positive value only at the vertex b , and zero at all other vertices. In this way, each hidden neuron behaves as a *soft vertex indicator* localized at b which is a continuous (differentiable) approximation of the indicator function of a Boolean vertex (corner) of the hypercube.

The Boolean function is then reconstructed by aggregating these localized contributions:

$$\text{NN}_f(x) = \sum_{b \in \mathcal{C}} \phi_b(x).$$

Since the supports of the neurons are disjoint over $\{0, 1\}^k$, the output coincides exactly with $f(x)$ on all Boolean inputs.

This construction shows that any Boolean function over a small number of variables can be implemented by a shallow ReLU network whose width equals the number of satisfying assignments of the function. This can be defined as the corner function is given in **Definition 5**.

Definition 5 (Corner Function [10]). Let $b = (b_1, \dots, b_k) \in \{0, 1\}^k$ be an active corner. The associated corner function is defined as

$$\text{corner}_{c, b_1, \dots, b_k}(x) = \text{ReLU} \left(\sum_{i: b_i=1} x_i + \sum_{i: b_i=0} (1 - x_i) - k + c \right),$$

for any choice of $0 < c \leq 1$.

Remark 1. For any input $x \in \{0, 1\}^k$, the expression inside the ReLU evaluates to c if $x = b$, and to a non-positive value otherwise. As a result, the corner function outputs a positive value only at the active corner b , while all other Boolean inputs are mapped to zero. This construction therefore isolates a single vertex of the Boolean hypercube, and extends naturally to real-valued inputs $x \in \mathbb{R}^k$.

For a Boolean function f with more than one active corner, its natural implementation is defined as the sum of the corner functions associated with all its active corners as:

Definition 6 (Sum of Corners [10]). Let $f_C : \{0, 1\}^k \rightarrow \{0, 1\}$ be a Boolean function, with active corners

$$\mathcal{C} = \{b = (b_1, \dots, b_k) \in \{0, 1\}^k : f_C(b) = 1\}.$$

The continuous generalization of f_C is defined as the sum of its corner functions over all active corners:

$$\Sigma_{\mathcal{C}}(x) = \left(\sum_{b \in \mathcal{C}} \text{corner}_{c, b_1, \dots, b_k}(x) \right).$$

Remark 2 (Complexity Bound). The natural implementation of f_C requires a single hidden layer containing $|\mathcal{C}|$ ReLU neurons, one for each active corner. In the worst case, $|\mathcal{C}| \leq 2^k$, yielding a network of size $O(2^k)$ and depth 2. For small constant values of k (e.g., $k = 8$ for AES S-boxes), this construction results in a compact and efficient ReLU-based neural network.

Finally, we define the *natural implementation* of a vectorial Boolean function

$$f : \{0, 1\}^k \rightarrow \{0, 1\}^m$$

as the concatenation of the m sum-of-corners constructions, placed side by side in the DNN. Refer to [10] for this construction which describes how to implement a vectorial Boolean function $f : \{0, 1\}^k \rightarrow \{0, 1\}^m$ as a neural network with k input neurons,

$$\sum_{o=1}^m |\mathcal{C}_o|$$

neurons in the hidden layer, and m output neurons implementing the m corresponding sums of corner functions.

These natural implementations of Boolean functions over a small number of bits (or Boolean functions over a large number of bits that have only a small number of active corners) can be composed into arbitrarily complex structures, allowing any Boolean circuit to be implemented as a DNN. Note that both the summation of corner functions and the parallel evaluation of the m output coordinates can be absorbed into the linear mapping of the next layer. As a result, each such composition requires only one additional layer in the DNN and note that natural DNN implementations compute continuous distance-like functions rather than discrete Boolean logic, and real-valued inputs turn secret key bits into observable geometric boundaries.

4 Natural Implementation of AES

The Advanced Encryption Standard (AES) [24] is a block cipher that encrypts 128-bit plaintexts using secret keys of length 128, 192, or 256 bits. The 128-bit key variant, denoted *AES-128*, applies 10 iterations of a round function to the internal state, which is represented as a 4×4 matrix of bytes.

Each round function is composed of four operations:

- **AddRoundKey (ARK)**: a bitwise XOR between the state and the round key;
- **SubBytes (SB)**: a non-linear byte-wise substitution layer;
- **ShiftRows (SR)**: a circular rotation applied to the rows of the state;
- **MixColumns (MC)**: a linear transformation consisting of a matrix multiplication applied independently to each column of the state. This operation is omitted in the final round.

The round keys are derived from the master key using the AES key schedule algorithm. Note that the first round key rk^1 is equal to the master key k . Consequently, an adversary who is able to recover the first round key can trivially derive all subsequent round keys. A high-level description of the implementation for AES-128 is given in **Algorithm 1**.

Algorithm 1: $NN_{AES_k}(p)$

Require: A 128-bit plaintext p and a 128-bit key k
Ensure: 128-bit ciphertext

```

1:  $(rk^1, \dots, rk^{11}) \leftarrow \text{AES\_key\_schedule}(k)$  // Derive round keys from AES key schedule
2:  $x \leftarrow NN_{ARK}(p, rk^1)$ 
3: for  $i = 1$  to 9 do
4:    $x \leftarrow NN_{SB}(x)$ 
5:    $x \leftarrow NN_{MC}(x)$ 
6:    $x \leftarrow NN_{ARK}(x, rk^{i+1})$ 
7: end for
8:  $x \leftarrow NN_{SB}(x)$ 
9:  $x \leftarrow NN_{SR}(x)$ 
10:  $x \leftarrow NN_{ARK}(x, rk^{11})$ 
11: return  $x$ 
```

All Boolean functions involved in the implementation are defined using $NN(\cdot, c)$, for a fixed-value c chosen *a priori* that is used to describe the different variations for key recovery attacks.

Bitwise XOR: $NN_{XOR}(x_1, x_2)$ In our natural implementation, the XOR operation is realized as a neural mapping $NN_{XOR}(x_1, x_2) = x_1 \oplus x_2$ implemented via the soft-corner construction defined in **Eq. (1)**. Byte-wise and 128-bit XORs are obtained by parallel concatenation of the bitwise network, denoted NN_{XOR_8} and $NN_{XOR_{128}}$, respectively. Higher-arity XORs are constructed by iterated composition of the basic bitwise XOR network.

AddroundKey: $NN_{ARK}(x, rk^i)$ The AddRoundKey operation is realized as a natural neural implementation of bitwise XOR between the 128-bit state p and the 128-bit round key k . Formally, it is implemented as $NN_{ARK}(p, k) = NN_{XOR_{128}}(p, k)$, that is, a parallel concatenation of 128 instances of the bitwise XOR network applied independently to each bit position.

SubBytes: $\text{NN}_{\text{SB}}(x)$ The SubBytes operation applies the AES S-box independently to each byte of the 128-bit state. Each 8-bit S-box is realized as the concatenation of eight natural Boolean networks, one for each output bit. Each output bit network is constructed as a sum of corner functions over the 8-dimensional Boolean cube corresponding to the active entries of the S-box truth table. Since the AES S-box is balanced, each output bit has 128 active corners, resulting in eight parallel corner-sum networks per byte.

Shift Rows and MixColumns: $\text{NN}_{\text{MC}}(x)$ The ShiftRows transformation applies a left rotation by predetermined i bytes for each row i . The MixColumns transformation of AES operates independently on each 32-bit column of the state. Let $x = (x_1, \dots, x_{32})$ denote one column interpreted as four bytes $a_0, b_0, c_0, d_0 \in \{0, 1\}^8$. In the binary AES specification, MixColumns applies the linear transformation

$$\begin{pmatrix} a_1 \\ b_1 \\ c_1 \\ d_1 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} a_0 \\ b_0 \\ c_0 \\ d_0 \end{pmatrix}$$

, where multiplication is performed in $\text{GF}(2^8)$ and addition corresponds to bitwise XOR. Each component of MixColumns is expressed as a composition of : Finite-field multiplication by constants 2 and 3, and 8-bit XOR operations.

Both operations are Boolean mappings over at most 8 bits and are therefore implemented using the Sum-of-Corners construction described in **Section 3**. Concretely, the neural MixColumns column transformation is defined as:

$$\begin{aligned} a_1 &= \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(a_0), c_0, d_0, \text{NN}_{\text{MUL3}}(b_0)), \\ b_1 &= \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(b_0), a_0, d_0, \text{NN}_{\text{MUL3}}(c_0)), \\ c_1 &= \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(c_0), a_0, b_0, \text{NN}_{\text{MUL3}}(d_0)), \\ d_1 &= \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(d_0), b_0, c_0, \text{NN}_{\text{MUL3}}(a_0)). \end{aligned}$$

Each subcomponent is a ReLU-based Sum-of-Corners network with corner parameter c . Although MixColumns is linear over $\text{GF}(2^8)$, its natural neural implementation becomes a piecewise-linear mapping over $\mathbb{R}^{32}$. When $c < 1$, the hidden ReLU neurons introduce separated linear regions corresponding to Boolean corners, which preserve the boundary structure exploited by the key recovery procedure. See **Algorithm 2** for its implementation.

Algorithm 2: $\text{NN}_{\text{MC}_j}(x)$

Require: A vector $x = (x_1, \dots, x_{32})$
Ensure: Output of the MixColumns operation on x

- 1: $a_0 \leftarrow (x_1, \dots, x_8)$
- 2: $b_0 \leftarrow (x_9, \dots, x_{16})$
- 3: $c_0 \leftarrow (x_{17}, \dots, x_{24})$
- 4: $d_0 \leftarrow (x_{25}, \dots, x_{32})$
- 5: $a_1 \leftarrow \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(a_0), c_0, d_0, \text{NN}_{\text{MUL3}}(b_0))$
- 6: $b_1 \leftarrow \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(b_0), a_0, d_0, \text{NN}_{\text{MUL3}}(c_0))$
- 7: $c_1 \leftarrow \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(c_0), a_0, b_0, \text{NN}_{\text{MUL3}}(d_0))$
- 8: $d_1 \leftarrow \text{NN}_{\text{XOR8}}(\text{NN}_{\text{MUL2}}(d_0), b_0, c_0, \text{NN}_{\text{MUL3}}(a_0))$
- 9: **return** (a_1, b_1, c_1, d_1)

5 Key Recovery

We describe a black-box key-recovery procedure that exploits the piecewise-linear structure of the natural DNN implementation. The attack assumes oracle access to

$$O(p) = \text{KeyRecovery}(\text{NN}_{\text{AES}}, \text{GenPairs}, \text{nbits}),$$

where the internal round keys are embedded into the network as described in **Algorithm 3** and **nbits** is the number of bits to be recovered, for brevity, we set **nbits** = 1.

Core Principle. The natural implementation of XOR with parameter $c < 1$ behaves as a separated linear region selector. If an input perturbation remains inside the same linear region, the network output remains invariant. If it crosses a ReLU boundary, the output changes. Therefore, equality of outputs under carefully constructed perturbations reveals the embedded key bit.

The recovery proceeds bit-by-bit:

1. Maintain a set of unresolved key bit positions.
2. For each trial, sample a random base plaintext.
3. For each unresolved position, generate structured plaintext pairs.
4. Test invariance of ciphertext outputs.
5. Fix the bit if exactly one candidate satisfies the invariance condition.

This process continues until all bits are recovered.

Algorithm 3: KeyRecovery($\text{NN}_{\text{AES}}, \text{GenPairs}, \text{nbits}$)

Require: A handle to a natural implementation of AES, $\text{NN}_{\text{AES}}(\cdot)$, for a secret key k ; a pair generation algorithm **GenPairs** for a variation parameter ε returning 2^{nbits} plaintext pairs, each corresponding to a candidate value for **nbits** consecutive key bits

Ensure: Candidate key k' for k , and number of base plaintexts used n_p

```

1: remainingPositions  $\leftarrow \{1, \dots, \frac{128}{\text{nbits}}\}$ 
2:  $n_p \leftarrow 0$ 
3: while remainingPositions  $\neq \emptyset$  do
4:   Select random base plaintext  $p$ 
5:    $n_p \leftarrow n_p + 1$ 
6:   for  $i \in \text{remainingPositions}$  do
7:     candidates  $\leftarrow \emptyset$ 
8:      $(x^0, x'^0), \dots, (x^{2^{\text{nbits}}}, x'^{2^{\text{nbits}}}) \leftarrow \text{GenPairs}(p, i)$ 
9:     for candidate key value  $v \in \{0, \dots, 2^{\text{nbits}} - 1\}$  do
10:       $C \leftarrow \text{NN}_{\text{AES}}(x^v)$ 
11:       $C' \leftarrow \text{NN}_{\text{AES}}(x'^v)$ 
12:      if  $C = C'$  then
13:        Add  $v$  to candidates
14:      end if
15:    end for
16:    if candidates contains a single value  $v$  then
17:       $k'_{i, \dots, i+\text{nbits}} \leftarrow v$ 
18:      Remove  $i$  from remainingPositions
19:    end if
20:  end for
21: end while
22: return  $k', n_p$ 

```

Comparison Between Classical Search and DNN-based Search. In a classical search setting, the adversary is restricted to binary queries over the domain $\mathbb{F}_2^k$ and interacts with the target algorithm solely through a Boolean oracle $f : \mathbb{F}_2^k \rightarrow \mathbb{F}_2$. Each query yields a single bit of information, and as a consequence, exhaustive or random search requires $\Theta(2^k)$ oracle evaluations on average to identify a hidden target.

In contrast, a DNN-based search operates on real-valued inputs in $\mathbb{R}^k$ and queries a deterministic ReLU-based neural network $F : \mathbb{R}^k \rightarrow \mathbb{R}$ that implements the same underlying functionality. Due to the piecewise-linear structure of ReLU networks, each real-valued query may reveal additional structural information, such as decision boundaries or local gradient behavior. This mismatch between the discrete oracle model and its continuous neural realization enables substantially more efficient search strategies, which in certain regimes allow the recovery of hidden components in time linear in k . The comparison between classical search and DNN-based search is summarized as **Table 1**.

Table 1: Comparison between classical search and DNN-based search

Item	Classical search	DNN-based search
Query domain	Binary vector space, $\mathbb{F}_2^k$	Real-valued vector space, $\mathbb{R}^k$
Oracle / black-box form	$f : \mathbb{F}_2^k \rightarrow \mathbb{F}_2$	Natural ReLU-DNN $F : \mathbb{R}^k \rightarrow \mathbb{R}$
Core resource	One evaluation per query	Reveal decision-boundary by gradient
complexity	$\Theta(2^k)$ (average)	$\Theta(k)$ (hidden-corner regime)
Source of speedup	None (random / exhaustive search)	Leakage of piecewise-linear structure
Interpretability	Standard cryptographic oracle model	Continuous-space realization

5.1 Generation of Pairs

The purpose of the pair-generation procedure is to construct two closely related plaintext inputs whose difference isolates the effect of a single candidate key bit in the first key-dependent XOR layer. Let $i \in \{0, \dots, 127\}$ denote an unresolved key-bit position. Given a base plaintext $p \in \{0, 1\}^{128}$, we generate structured pairs (x_v, x'_v) for $v \in \{0, 1\}$, such that the two inputs differ only in coordinate i , and the perturbation magnitude is controlled by a small parameter $\varepsilon > 0$.

Invariance Test. For each candidate bit value v , we evaluate $c_1 = O(x_v)$, $c_2 = O(x'_v)$. We then test the invariance condition $O(x_v) = O(x'_v)$. If the natural XOR layer with parameter $c < 1$ does not cross a ReLU boundary under the perturbation, the outputs remain identical. Otherwise, a boundary is crossed and the outputs differ.

Key Identification Principle. Under the XOR-first assumption, exactly one candidate value $v \in \{0, 1\}$ satisfies the invariance condition. This uniquely determines the correct key bit k_i . The following subsections describe two concrete constructions of the pair-generation mechanism.

5.1.1 GenPairs_{change}(p, i, ε) The change-based generation method constructs asymmetric perturbations around Boolean values. Given base plaintext p and bit index i , define:

$$(x_0, x'_0) = (p \mid p_i = 0, p \mid p_i = \varepsilon),$$

$$(x_1, x'_1) = (p \mid p_i = 1, p \mid p_i = 1 - \varepsilon).$$

Only the i -th coordinate is modified as described in **Algorithm 4** [10]. If the true key bit equals v , then perturbing within the linear region consistent with v does not cross a ReLU boundary, and the ciphertext remains unchanged. If the key bit differs, the perturbation crosses a boundary, and the ciphertext changes. Thus, invariance uniquely identifies the correct bit.

Algorithm 4: $\text{GenPairs}_{\text{change}}(p, i, \varepsilon)$

- 1: $(x^0, x'^0) \leftarrow ((p \mid p_i = 0), (p \mid p_i = \varepsilon))$
 - 2: $(x^1, x'^1) \leftarrow ((p \mid p_i = 1), (p \mid p_i = 1 - \varepsilon))$
 - 3: **return** $(x^0, x'^0), (x^1, x'^1)$
-

5.1.2 $\text{GenPairs}_{\text{sym}}(p, i, \varepsilon)$ The symmetric variant uses perturbations around Boolean boundaries:

$$(x_0, x'_0) = (p \mid p_i = -\varepsilon, p \mid p_i = +\varepsilon),$$

$$(x_1, x'_1) = (p \mid p_i = 1 - \varepsilon, p \mid p_i = 1 + \varepsilon).$$

This construction probes the behavior of the network on both sides of a potential ReLU boundary. If the key bit equals v , both perturbations lie in the same linear region, yielding identical ciphertext outputs. Otherwise, the perturbation straddles a boundary, and outputs differ. The symmetric method is numerically more stable near $c \rightarrow 1$, and empirically reduces the number of outer loops required for convergence as described in **Algorithm 5** [10].

Algorithm 5: $\text{GenPairs}_{\text{sym}}(p, i, \varepsilon)$

- 1: $(x^0, x'^0) \leftarrow ((p \mid p_i = -\varepsilon), (p \mid p_i = \varepsilon))$
 - 2: $(x^1, x'^1) \leftarrow ((p \mid p_i = 1 - \varepsilon), (p \mid p_i = 1 + \varepsilon))$
 - 3: **return** $(x^0, x'^0), (x^1, x'^1)$
-

Lemma 1 (Local Separability of Natural XOR). *Let the first key-dependent operation be a natural XOR layer with parameter $c < 1$. For sufficiently small $\varepsilon > 0$, exactly one of the two candidate perturbations remains within the same linear region of the ReLU network.*

Proof (Sketch). Consider the natural XOR network $\text{NN}_{\text{XOR}}(\cdot, \cdot; c)$ implemented by the Sum-of-Corners construction. For a fixed key bit $k_i \in \{0, 1\}$, the i -th XOR output is a piecewise-linear function of the scalar input x_i (with all other coordinates fixed to the base plaintext p). When $c < 1$, the ReLU activations associated with the two relevant corners of XOR induce two disjoint linear regions around $x_i \approx k_i$ and $x_i \approx 1 - k_i$. In particular, there exists a margin $\delta(c) > 0$ such that for all perturbations satisfying $0 < \varepsilon < \delta(c)$, the pair (x_{k_i}, x'_{k_i}) remains in the same linear region (no ReLU boundary is crossed), hence $\text{NN}_{\text{XOR}}(x_{k_i}; c) = \text{NN}_{\text{XOR}}(x'_{k_i}; c)$. Conversely, the alternative pair (x_{1-k_i}, x'_{1-k_i}) necessarily

straddles a boundary of the corresponding region, so at least one ReLU activation pattern changes, implying $\text{NN}_{\text{XOR}}(x_{1-k_i}; c) \neq \text{NN}_{\text{XOR}}(x'_{1-k_i}; c)$. Under the XOR-first assumption, this invariance (or non-invariance) at the first key-dependent layer propagates through the subsequent deterministic layers of the network, yielding $O(x_{k_i}) = O(x'_{k_i})$ and $O(x_{1-k_i}) \neq O(x'_{1-k_i})$. Therefore, exactly one candidate value $v \in \{0, 1\}$ satisfies the invariance test, uniquely identifying k_i .

6 Natural AES-128 Oracle and Bitwise Key Recovery

6.1 Overview

We describe a natural neural implementation of AES-128 where all Boolean components (SubBytes, MixColumns, and XOR) are implemented using a *Sum-of-Corners* ReLU network. The AES round keys are embedded inside the network, and the cipher is accessed only as a black-box oracle returning real-valued ciphertext bits. We then apply a bitwise key-recovery procedure that exploits the separated ReLU behavior when the corner parameter $c < 1$.

Algorithm 6: Natural AES-128 Encryption Oracle

Require: 128-bit key K , corner parameter $c \in (0, 1]$

Ensure: Black-box encryption oracle $\mathcal{O}(\cdot)$

```

1: Compute AES-128 round keys  $K^{(0)}, \dots, K^{(10)}$ 
2: Convert each round key into bit vectors
3:  $x \leftarrow \text{NN}_{\text{XOR}}(x, K^{(0)}, c)$ 
4: for  $r = 1$  to 9 do
5:    $x \leftarrow \text{NN}_{\text{SB}}(x, c)$ 
6:    $x \leftarrow \text{NN}_{\text{SR}}(x)$ 
7:    $x \leftarrow \text{NN}_{\text{MC}}(x, c)$ 
8:    $x \leftarrow \text{NN}_{\text{XOR}}(x, K^{(r)}, c)$ 
9: end for
10:  $x \leftarrow \text{NN}_{\text{SB}}(x, c)$ 
11:  $x \leftarrow \text{NN}_{\text{SR}}(x)$ 
12:  $x \leftarrow \text{NN}_{\text{XOR}}(x, K^{(10)}, c)$ 
13:
14: return  $x$ 
```

As described in [Section 2.2](#), we verify the correctness of our natural AES-128 encryption oracle by comparing its outputs with those of the standard Boolean implementation. For this validation, we use the well-known test vector provided in FIPS PUB 197 [\[24\]](#).

```

==Known Answer Test from our implementation==
PT : 00112233445566778899aabbccddeeff
KEY: 000102030405060708090a0b0c0d0e0f
CT Boolean: 69c4e0d86a7b0430d8cdb78070b4c55a
CT Neural : 69c4e0d86a7b0430d8cdb78070b4c55a
MATCH: True
```

6.2 Operational Explanation

The attack exploits the following structural property as described in [Algorithm 7](#):

- [Step 1] When $c < 1$, the corner-sum implementation of XOR behaves as a *piecewise-linear separator*.
- [Step 2] If a plaintext bit is perturbed inside the linear region consistent with the embedded key bit, the ciphertext does not change.
- [Step 3] If it crosses a ReLU boundary, the ciphertext changes.
- [Step 4] Therefore, equality of ciphertext outputs under small perturbations reveals the hidden key bit.

As mentioned before, unlike classical differential cryptanalysis, the attack:

- Operates in real space,
- Uses infinitesimal perturbations instead of bit flips,
- Treats the cipher purely as a DNN oracle,
- Recovers the full 128-bit key without accessing intermediate states.

Algorithm 7: Bitwise Key Recovery

Require: Black-box oracle $\mathcal{O}$, perturbation $\varepsilon > 0$
Ensure: Recovered key bits $\hat{k} \in \{0, 1\}^{128}$

- 1: Initialize $\hat{k}_i \leftarrow -1$ for all i
- 2: $\mathcal{R} \leftarrow \{0, \dots, 127\}$
- 3: **while** $\mathcal{R} \neq \emptyset$ **do**
- 4: Sample random base plaintext $p \in \{0, 1\}^{128}$
- 5: **for** each $i \in \mathcal{R}$ **do**
- 6: Construct pairs $(x_0, x'_0), (x_1, x'_1)$ using `GenPairs()` :
- 7: **for** $v \in \{0, 1\}$ **do**
- 8: Compute $c_1 = \mathcal{O}(x_v)$
- 9: Compute $c_2 = \mathcal{O}(x'_v)$
- 10: **if** $c_1 = c_2$ **then**
- 11: Candidate bit v
- 12: **end if**
- 13: **end for**
- 14: **if** exactly one candidate **then**
- 15: $\hat{k}_i \leftarrow v$
- 16: remove i from $\mathcal{R}$
- 17: **end if**
- 18: **end for**
- 19: **end while**
- 20:
- 21: **return** $\hat{k}$

6.3 Sum-of-Corners Neural Implementation of Boolean Functions

We describe a neural construction that implements any Boolean mapping

$$f : \{0, 1\}^k \rightarrow \{0, 1\}^m$$

using a single hidden layer of ReLU neurons.

Construction. Let the Boolean mapping be represented by a truth-table vector

$$\text{mapping} \in \{0, \dots, 2^m - 1\}^{2^k},$$

where each entry encodes the m -bit output of f for a given input index. For each output coordinate $o \in \{0, \dots, m-1\}$, define the set of *active corners*

$$B_o = \{b \in \{0, 1\}^k : \text{the } o\text{-th output bit of } f(b) = 1\}.$$

Each corner b corresponds to one hidden neuron.

Verification of the Corner Set. For a Boolean mapping $f : \{0, 1\}^k \rightarrow \{0, 1\}^m$, Definition 6 represents each output bit f_o as a sum of corner functions over its active corner set

$$B_o = \{b \in \{0, 1\}^k \mid f_o(b) = 1\}.$$

In our implementation, the class **SumOfCornersBoolean** explicitly computes these sets by extracting all inputs b such that the o -th output bit of the truth table equals 1. The printed output of the network therefore directly lists the elements of B_o for each output coordinate.

Formally, the neural realization of f_o is given by

$$\text{NN}_{f_o}(x) = \sum_{b \in B_o} \text{ReLU}\left(\frac{\langle x, b \rangle + \langle 1 - x, 1 - b \rangle - k + c}{c}\right),$$

which equals 1 if and only if $x \in B_o$, and equals 0 otherwise. Hence, the printed corner sets constitute an explicit verification that the neural network exactly implements the Boolean mapping defined by the given truth table.

The explicit sum-of-corners representation of the AES S-box requires a substantial amount of space, since each of its eight output bits has 128 active corners in the 8-dimensional Boolean cube. To provide a clearer and more compact illustration of the construction, we instead present a simple 3-bit bijective S-box as an example.

Example of a 3-bit Bijective S-box: Consider the 3-bit S-box defined by the permutation

$$S = [7, 6, 4, 3, 2, 1, 0, 5],$$

that is,

$$S(0) = 7, S(1) = 6, S(2) = 4, S(3) = 3, S(4) = 2, S(5) = 1, S(6) = 0, S(7) = 5.$$

Its Boolean representation (in binary output form) is given by

x	$S(x)$	x	$S(x)$
0	111	4	010
1	110	5	001
2	100	6	000
3	011	7	101

Since each output value in $\{0, 1, \dots, 7\}$ appears exactly once, the mapping is bijective and therefore defines a permutation on $\{0, 1\}^3$.

For each output bit S_o , the active corner set

$$B_o = \{x \in \{0, 1\}^3 \mid S_o(x) = 1\}$$

contains exactly four elements, namely:

$$B_0 = \{0, 1, 2, 7\},$$

$$B_1 = \{0, 1, 3, 4\},$$

$$B_2 = \{0, 3, 5, 7\}.$$

Thus each coordinate Boolean function is balanced, with Hamming weight 4, which corresponds to half of the Boolean cube $\{0, 1\}^3$.

6.4 Bitwise Key Recovery

In all experiments, AES-128 was modeled as a DNN using corner-sum implementations for XOR, S-box, and MixColumns. The attack targets the first round key and recovers all 128 key bits. Two independent experiments were developed:

- Exp.1: Large-scale Monte-Carlo evaluation over 1,000 random keys.
- Exp.2: Systematic screening and precision sweep over (c, ε) .

6.4.1 Using $\text{GenPairs}_{\text{change}}$ We implemented a natural neural representation of AES-128 using sum-of-corners Boolean networks and evaluated a DNN-based key-recovery attack using the $\text{GenPairs}_{\text{change}}$ strategy.

Exp.1: Monte-Carlo Evaluation over 1,000 Random Keys We first evaluated the attack over 1,000 independently sampled uniform random AES-128 keys using parameters

$$c = 0.5, \quad \varepsilon = 0.01.$$

For each key, the attack was executed once. All 1,000 keys were recovered correctly, with Success Rate = $1,000/1,000 = 1.00$. For each run, the recovered key matched the true key exactly, confirming deterministic correctness under these parameters. This demonstrates that the DNN-based key-recovery attack succeeds with overwhelming probability in the tested regime. The below is the final printout for selected key randomly and its recovered key at 999th and 1,000th run, respectively.

```
=== Run 999/1000 ===
Selected key (hex)= a32e43d976ff8c8984302e168b31724d
recovered key(hex)= a32e43d976ff8c8984302e168b31724d
```

```
=== Run 1000/1000 ===
Selected key (hex)= 4bafa0feccf59b66b720abc37cd6caec
recovered key(hex)= 4bafa0feccf59b66b720abc37cd6caec
```

Exp.2: Parameter Screening over (c, ε) We next performed a grid search over parameters:

$$c \in \{0.1, 0.2, \dots, 0.9\}, \quad \varepsilon \in \{0.001, 0.01, 0.1\}.$$

For each parameter pair, 20 independent trials were executed.

Observation 1 (Success Probability). All tested parameter combinations achieved

$$\text{Success Rate} = 20/20 = 1.00.$$

Observation 2 (Loop Complexity). The average number of outer loops increases as:

- c approaches 1, and
- ε becomes very small.

For example:

$$\begin{aligned} c = 0.1, \varepsilon = 0.1 &\Rightarrow \text{avg loops} \approx 1.0, \\ c = 0.8, \varepsilon = 0.001 &\Rightarrow \text{avg loops} \approx 8.1. \end{aligned}$$

Thus, smaller ε improves numerical precision but increases iteration cost.

Precision Sweep for High- c Regime We further examined the high- c region:

$$c \in \{0.90, 0.92, 0.94, 0.96, 0.98\}, \quad \varepsilon \in \{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}\}.$$

Again, each configuration was tested over 20 independent trials.

Result. All configurations achieved:

$$\text{Success Rate} = 20/20 = 1.00.$$

However, the loop complexity increases noticeably for small ε :

$$c = 0.98, \varepsilon = 10^{-4} \Rightarrow \text{avg loops} \approx 8\text{--}9.$$

This confirms that numerical sharpness of the corner activation controls convergence speed but does not affect correctness.

Main Experimental Findings Our experiments establish the following:

- The DNN-based key recovery attack succeeds with empirical probability 1 across all tested parameter regimes.
- The attack remains robust over 1,000 independent AES-128 keys.
- Parameter (c, ε) affects convergence speed but not correctness.
- The observed average outer-loop complexity is small (typically between 1 and 8 iterations).

These results strongly suggest that AES-128, when implemented as a natural sum-of-corners neural network, is vulnerable to a practical linear-time key-recovery attack.

6.4.2 Using $\text{GenPairs}_{\text{sym}}$ We experimentally evaluated the DNN-based key recovery attack on natural AES-128 using the $\text{GenPairs}_{\text{sym}}$ strategy. All experiments were performed with 128-bit random master keys, and success was measured as full recovery of the 128-bit round key.

Exp3: Monte Carlo over 100 Random-Key We additionally performed 1,000 independent trials with uniformly random AES-128 keys ($c = 0.5, \varepsilon = 0.01$). The below is the final printout for selected key randomly and its recovered key at 999th and 1,000th run, respectively.

=== Run 999/1000 ===

Selected key (hex)= b4915ba3f363a186a2c8aff99667056c

recovered key(hex)= b4915ba3f363a186a2c8aff99667056c

=== Run 1000/1000 ===

Selected key (hex)= 621b5b64f796cf12d97859ca50ff9517

recovered key(hex)= 621b5b64f796cf12d97859ca50ff9517

In all 1,000 runs, the recovered key exactly matched the true key, yielding an empirical success rate of

$$\Pr[\text{success}] = 1.0 \quad (1,000/1,000).$$

Exp4: Parameter Screening over (c, ε) In this experiment, we explored a coarse grid of parameters $c \in \{0.1, \dots, 0.9\}$ and $\varepsilon \in \{10^{-3}, 10^{-2}, 10^{-1}\}$, with 20 repetitions per parameter pair.

For all tested (c, ε) combinations, the attack achieved a 100% success rate (20/20). However, the average number of outer recovery loops increased with larger c and smaller ε .

Notably:

- For small c (e.g., $c = 0.1$), $\text{avg_loops} \approx 1.0$ – 1.3 .
- For larger c (e.g., $c = 0.9$) and $\varepsilon = 10^{-3}$, $\text{avg_loops} \approx 7.3$, with a maximum of 14 loops.

This confirms that while correctness is robust across a wide parameter range, convergence speed depends significantly on (c, ε) .

Precision Sweep Phase. In the second phase, we refined the parameter region around $c \in [0.9, 0.98]$ and $\varepsilon \in \{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 10^{-2}\}$.

Again, the success rate remained 100% for all tested values. However, a clear trend was observed:

- Smaller ε leads to higher average loop counts.
- Increasing ε to 10^{-2} reduces avg_loops to approximately 3–6.
- Very small ε (e.g., 10^{-4}) results in avg_loops around 7–9.

Key Findings Across all experiments, the DNN-based key recovery attack achieves deterministic full key recovery for natural AES-128 implementations under $\text{GenPairs}_{\text{sym}}$. The parameters (c, ε) mainly influence convergence speed, but not correctness within the tested region. These results strongly support the claim that natural neural implementations of AES constitute a structurally exploitable attack surface, allowing practical linear-time key recovery.

6.4.3 Why no failure in our experiment The theoretical separation between correct and incorrect hypotheses may vanish when the normalized corner activation

$$\frac{\text{match} - k + c}{c}$$

becomes excessively flat. In particular, when c approaches 1, the slope of the ReLU corner function decreases, and the distinction between exact matches and near-matches is attenuated. However, in practice, this theoretical flattening does not eliminate separability for three reasons:

- **Role of ε .** The perturbation parameter ε introduces controlled asymmetry in pair generation, ensuring that correct and incorrect key hypotheses produce statistically distinguishable outputs. Even when the corner slope is small, ε induces measurable bias.
- **Floating-Point Resolution.** The DNN implementation operates in floating-point arithmetic. The piecewise-linear ReLU function preserves fine-grained numerical differences, preventing exact collapse of activations. Thus, practical computations maintain effective piecewise separation.
- **XOR Depth Amplification.** AES consists of multiple nonlinear and linear layers. Repeated XOR and S-box compositions amplify small deviations introduced in early layers. Consequently, even minor activation differences accumulate across rounds, leading to statistically significant divergence between correct and incorrect key candidates.

Therefore, although the analytical expression suggests that separation may vanish for large c , empirical results confirm that effective distinguishability is preserved through perturbation bias, numerical precision, and depth amplification effects.

7 Round-Key Recoverability for AES-192

We present a comparison of the key expansion of AES-128, AES-192 and AES-256, as summarized in **Table 2**.

Table 2: Comparison of key expansion in AES-128, AES-192, and AES-256

Algorithm	AES-128	AES-192	AES-256
Key Size(bits)	128	192	256
# of Key(Word), Nk	4	6	8
# of Rounds, Nr	10	12	14
Total Round Keys(Word)	11	13	15
Total Expanded Keys(Word)	44	52	60
Initial Words, $(w_0 \dots w_{Nk-1})$	$w_0 \dots w_3$	$w_0 \dots w_5$	$w_0 \dots w_7$
When $g(w_{i-1})$ Applied	$i \bmod 4 = 0$	$i \bmod 6 = 0$	$i \bmod 8 = 0$
Additional SubWord Step	No	No	Yes, if $i \bmod 8 = 4$
rk_0 = Master Key?	Yes	First 128 bits	First 128 bits
Diffusion Speed	Fast	Moderate	Slower
Nonlinearity Injection	Per 4 words	Per 6 words	Per 8 words + extra

7.1 Threat Model Revisited

Black-box model We assume standard encryption-oracle access to AES. An adversary may query a black-box oracle

$$\mathcal{O}(p) = E_K(p),$$

for chosen plaintexts $p \in \{0, 1\}^{128}$ and observe the corresponding ciphertexts. No internal state, intermediate values, or implementation artifacts are revealed.

Synthetic intermediate-state model For structural analysis, we additionally consider a synthetic oracle that exposes a chosen intermediate state as the input to a reduced encryption procedure. For a round index r , the oracle takes an input z interpreted as the state immediately before the AddRoundKey operation of round r , applies

$$z \mapsto z \oplus rk_r,$$

and then executes the remaining rounds of AES. This oracle is synthetic in that it is constructed to isolate a specific key-dependent layer and does not correspond to the standard black-box AES interface.

7.2 Why rk_0 Is Recoverable in the Black-box Model

The symmetry-based DNN bit test relies on the fact that the first key-dependent operation encountered by an adversary-controlled input is a natural XOR layer of the form

$$x \mapsto x \oplus k,$$

implemented via a natural neural representation.

In AES, the initial AddRoundKey satisfies this condition:

$$x_0 = p \oplus rk_0.$$

Since plaintext perturbations directly reach this XOR layer without passing through any non-linear transformation, the symmetry invariance exploited by the DNN test holds. Consequently, the bits of rk_0 can be recovered under standard black-box oracle access.

7.3 Why rk_1 Is Not Directly Recoverable in the Black-box Model

After the initial AddRoundKey, Round 1 of AES applies public non-linear and diffusion layers before the next key addition:

$$x_1 = \text{MC}(\text{SR}(\text{SB}(x_0))) \oplus rk_1.$$

Therefore, adversary-controlled plaintext perturbations must first pass through the SubBytes non-linearity before reaching the XOR with rk_1 . The symmetry-based bit test assumes that the first key-dependent operation is XOR; this structural property is violated for rk_1 . As a result, the invariance condition underlying the bit test does not hold in general, and rk_1 cannot be recovered directly using the same black-box methodology.

7.4 Why the Synthetic Oracle Enables Recovery of rk_1

In the synthetic model, we explicitly expose the input to the AddRoundKey of round r . The resulting computation begins with

$$z \mapsto z \oplus rk_r,$$

followed by the remaining AES rounds. This restores the required condition that the first key-dependent layer is XOR. Hence, the symmetry-based bit test applies again, allowing iterative recovery of rk_r for arbitrary r . By construction, the recovered key equals the true round key, since the XOR operation uniquely determines the key bits under the symmetry invariance condition.

Under standard black-box access, the natural DNN symmetry test directly targets the initial AddRoundKey and thus recovers rk_0 . Recovering rk_1 requires additional access (synthetic, white-box, or leakage), because the key-dependent XOR of round 1 is preceded by the public S-box nonlinearity.

Unlike the key recovery behavior observed in AES-128, our implementation results demonstrate that the `GenPairssym` approach consistently recovered the external 192-bit master key, leveraging synthetic oracles derived from rk_0 , rk_1 , and rk_2 in all 1,000 trials. The below is the partial printout.

[Experiment 999]

```
Random master key      = 62816b06fe8860b4a868fc09536880b957f24eb7140ac5c4
== Using genpairs_sym !=
== Using genpairs_sym !=
== Using genpairs_sym !=
Recovered master key    = 62816b06fe8860b4a868fc09536880b957f24eb7140ac5c4
Result                  = SUCCESS
.
```

[Experiment 1000]

```
Random master key      = f9baa5c55b42477af0024b9643bd011709da012352a1a9a0
```

Algorithm 8: Bit-wise Natural Key Recovery–AES192-symm

Require: Oracle $\mathcal{O}$, bit length n , perturbation ε , max loops $L_{\max}$, seed s

Ensure: $\hat{k} \in \{-1, 0, 1\}^n$, number of loops L

```
1: Initialize RNG with seed  $s$ 
2: remaining  $\leftarrow \{0, \dots, n-1\}$ 
3:  $\hat{k} \leftarrow (-1, \dots, -1)$ 
4:  $L \leftarrow 0$ 
5: while remaining  $\neq \emptyset$  and  $L < L_{\max}$  do
6:    $L \leftarrow L + 1$ 
7:   Sample  $p \leftarrow \{0, 1\}^n$ 
8:   new_remaining  $\leftarrow \emptyset$ 
9:   for all  $i \in$  remaining do
10:    cand  $\leftarrow \emptyset$ 
11:    for  $v \in \{0, 1\}$  do
12:      Construct  $(x_v, x'_v)$  by symmetric perturbation at index  $i$ 
13:      if  $\mathcal{O}(x_v) = \mathcal{O}(x'_v)$  then
14:        cand  $\leftarrow$  cand  $\cup \{v\}$ 
15:      end if
16:    end for
17:    if |cand| = 1 then
18:       $\hat{k}[i] \leftarrow$  unique element of cand
19:    else
20:      new_remaining  $\leftarrow$  new_remaining  $\cup \{i\}$ 
21:    end if
22:  end for
23:  remaining  $\leftarrow$  new_remaining
24: end while
25:
26: return  $(\hat{k}, L)$ 
```

```
== Using genpairs_sym !=
== Using genpairs_sym !=
== Using genpairs_sym !=
Recovered master key = f9baa5c55b42477af0024b9643bd011709da012352a1a9a0
Result               = SUCCESS
```

In contrast, the $\text{GenPairs}_{\text{change}}$ approach failed to achieve successful recovery. The essential difference between $\text{GenPairs}_{\text{sym}}$ and $\text{GenPairs}_{\text{change}}$ for this discrepancy is summarized in Table 3.

Table 3: Essential difference between $\text{GenPairs}_{\text{sym}}$ and $\text{GenPairs}_{\text{change}}$

Item	$\text{GenPairs}_{\text{sym}}$	$\text{GenPairs}_{\text{change}}$
Boundary Crossing Strength	Strong	Weak
Bit Determination Speed	Fast	Slow
Outer Loop Size	Small	Large
Number of Oracle Calls	Few	Very Large
Overall Runtime	Fast	Very Slow

8 Applicability of the Synthetic Oracle Search to AES-256

8.1 Synthetic Oracle Requirement

The symmetry-based DNN key recovery method applies when the first key-dependent operation encountered by an adversary-controlled input is a natural XOR layer of the form

$$x \mapsto x \oplus rk_r.$$

In the synthetic intermediate-state model, we explicitly construct an oracle that exposes the state immediately before the AddRoundKey operation of round r . The oracle is defined as

$$\mathcal{O}_r(z) = E_K^{(r \rightarrow Nr)}(z \oplus rk_r),$$

where $E_K^{(r \rightarrow Nr)}$ denotes execution of AES from round r to the final round.

Since the computation begins with

$$z \mapsto z \oplus rk_r,$$

the required XOR-first condition is restored, and the symmetry-based bit test applies.

8.2 Extension from AES-192 to AES-256

The round function of AES-256 is identical to that of AES-192:

$$\text{SB} \rightarrow \text{SR} \rightarrow \text{MC} \rightarrow \oplus rk_r.$$

Although AES-256 differs in:

- key size ($Nk = 8$ instead of 6),
- number of rounds ($Nr = 14$ instead of 12),
- and the additional SubWord step in the key expansion,

these differences affect only the key schedule and not the structure of the round function. Because the synthetic oracle isolates the AddRoundKey layer, the additional SubWord operation in the AES-256 key expansion does not interfere with the symmetry-based recovery.

8.3 Computational Cost

For each round key:

- Bitwise recovery requires $O(128)$ tests.
- The number of outer loops is comparable to AES-192.
- AES-256 incurs two additional rounds, adding only minor overhead.

Thus, total complexity scales approximately with

$$128 \cdot (Nr + 1),$$

and AES-256 requires roughly $\frac{15}{13} \approx 1.15$ times the work of AES-192.

8.4 Key Findings

The synthetic intermediate-state oracle restores the XOR-first condition required by the symmetry-based DNN attack. Since AES-256 preserves the same round structure as AES-192, the synthetic search extends directly and enables recovery of all round keys under the synthetic threat model.

Importantly, the vulnerability exploited by the natural DNN attack is independent of:

- key size (128, 192, or 256 bits),
- number of rounds,
- key schedule complexity.

It depends solely on exposure of a natural XOR layer as the first key-dependent transformation. Similar to the key recovery results for AES-192, our implementation demonstrates that the Pairgen-sym approach consistently recovered the external 256-bit master key, leveraging synthetic oracles derived from rk0, rk1, and rk2 in all 1,000 trials. In contrast, the Pairgen-change approach failed to achieve successful recovery again. The below is the final printout to recover master key of AES-256 with success.

```
true key = cde9f303999caee236dcb97db36c70d96eab9e82e816938058c4d41b1fd322f
== Using genpairs_sym !=
recovered rk0 = cde9f303999caee236dcb97db36c70d loops= 1
== Using genpairs_sym !=
recovered rk1 = 96eab9e82e816938058c4d41b1fd322f loops= 1
== Using genpairs_sym !=
recovered rk2 = 98cae6cb01564865223b83f2f90d44ff loops= 1
true rk0      = cde9f303999caee236dcb97db36c70d
true rk1      = 96eab9e82e816938058c4d41b1fd322f
true rk2      = 98cae6cb01564865223b83f2f90d44ff
[check] rk0=True, rk1=True, rk2=True
recovered master key= cde9f303999caee236dcb97db36c70d96eab9e82e816938058c4d41b1fd322f
true master key     = cde9f303999caee236dcb97db36c70d96eab9e82e816938058c4d41b1fd322f
[master check] = True
```

9 Related Work

Boolean functions, cryptography and neural network In 1943, McCulloch and Pitts [21] showed that networks of idealized neurons can be modeled as logical operators, proving that neural activity can in principle compute any function expressible in formal logic—laying the foundations of neural networks and computational neuroscience. In 1958, Rosenblatt [25] introduced the perceptron—a mathematically defined, probabilistic model of a simplified neural unit that can learn to store and organize information through adjustable weights, laying practical foundational work for artificial neural networks. Minsky and Papert’s Perceptrons [22] rigorously analyzed the computational capabilities and limitations of single-layer perceptron networks, showing that simple perceptrons cannot solve certain fundamental pattern-classification problems (e.g., XOR), which deeply influenced subsequent research directions in neural network theory. Cybenko’s 1989 paper [7] proves that feedforward neural networks with a single hidden layer and any continuous sigmoidal activation function can uniformly approximate any continuous function on a compact domain to arbitrary accuracy, establishing a foundational form of the universal approximation theorem. Hornik, et al. [12] proved that standard multilayer feedforward neural

networks with at least one hidden layer and suitable activation functions can approximate any measurable function on a compact domain to arbitrary accuracy, establishing a general form of the universal approximation theorem. While it has been known for a long time that neural networks can be used to build any Boolean circuits, the implementation of cryptographic primitives into neural network has received very little attention.

Recently, Goldwasser et al. [11] proposed a backdoor construction for neural networks based on a cryptographic signature algorithm, implemented as described in [22], with a non-continuous step activation function. This idea of adding cryptographic backdoors to neural networks was also extended to large language models in [9] and [13]. Draguns et al. [9] introduces a novel class of cryptographically constructed backdoors in transformer-based language models that are unelicitable — meaning they cannot be triggered or detected even with full white-box access — and demonstrates both theoretical and empirical evidence that such backdoors can evade state-of-the-art mitigation and pre-deployment detection strategies, fundamentally challenging current AI safety evaluation methods. Kalavasis et al. [13] proves—under standard cryptographic assumptions—that even with full white-box access, neural networks and language models released via indistinguishability obfuscation can contain provably undetectable and non-replicable backdoors, demonstrating a fundamental limitation of current model inspection and safety auditing methods.

Finally, Bullini et al. [2] analyzes the theoretical limits of neural networks in black-box cryptanalysis of symmetric block ciphers. By modeling a cipher as a vectorial Boolean function, the authors show that accurately learning random Boolean components of n variables requires on the order of 2^n training samples, making full emulation infeasible for modern ciphers such as AES. They explain that previously reported neural attacks succeed mainly in reduced-round settings where output bits depend on only a few inputs. Overall, the work demonstrates that neural networks do not provide a practical advantage in realistic black-box cryptanalysis and clarifies the fundamental boundaries of such approaches.

Neural Network for Cryptography The dual problem of building cryptography using neural networks has been more popular. Kanter et al. [14] proposes a novel neural-network-based key exchange protocol in which two parties securely generate a shared secret by synchronizing their neural network weights through mutual learning, making eavesdropping computationally infeasible despite full knowledge of the protocol. However Klimov et al. [15] analyzes this key exchange protocol [14], explains why mutual learning leads the two honest parties to synchronize on a shared key, but ultimately demonstrates via genetic, geometric, and probabilistic attacks that an eavesdropper can reliably recover the key, rendering the scheme completely insecure.

Abadi and Anderson [1] investigates whether neural networks can learn cryptographic behavior autonomously by training communicating agents (Alice and Bob) to protect information from an adversarial neural network (Eve) without prescribing any explicit cryptographic algorithms; using end-to-end adversarial training, the authors show that the networks can discover and apply effective forms of encryption, decryption, and selective information hiding, demonstrating that confidentiality properties can emerge purely from learning objectives defined in terms of an adversary’s capabilities rather than hand-designed cryptographic primitives.

Coutinho et al. [6] analyzes the security of cryptographic schemes generated by Adversarial Neural Cryptography (ANC), highlighting that prior neural-network-based approaches often lack provable security understanding, and demonstrates that while AI agents can autonomously learn a provably secure scheme such as the one-time pad under appropriate training conditions and adversarial strength, weaker adversaries tend to lead the system

toward insecure encryption, revealing both the limitations and necessary conditions for learning truly secure cryptographic algorithms.

Cryptography for Neural Network Mann et al. [19] surveys recent progress in secure neural network inference, addressing the need to protect sensitive inputs and model parameters during inference, and systematically compares existing approaches—such as those based on homomorphic encryption and secure multi-party computation—in terms of efficiency, security, accuracy, and practicality, while also identifying key open challenges that must be overcome for large-scale real-world deployment.

10 Conclusion

We have demonstrated that AES-128, AES-192, and AES-256 admit deterministic linear-time master-key recovery when implemented using the natural sum-of-corners ReLU construction and exposed to real-valued oracle queries. Our attack operates in a strictly defined threat model in which the adversary interacts with a correct neural implementation of AES over $\mathbb{R}^{128}$, rather than the classical Boolean oracle over $\mathbb{F}_2^{128}$.

The central technical insight is that the natural ReLU realization of the AddRoundKey layer induces a partition of the input space into separated linear regions whose boundaries depend explicitly on secret key bits. Under symmetric perturbations, these regions exhibit a local separability property: exactly one candidate key hypothesis preserves linear-region membership. This structural invariant allows bitwise recovery of round keys using only equality tests on oracle outputs, yielding empirical complexity $O(128R)$ for R rounds. Our results do not contradict the standard security of AES in the classical black-box model. Rather, they expose a fundamental model mismatch between discrete cryptographic security definitions and continuous neural implementations. In the Boolean setting, each query reveals a single bit of information. In contrast, a real-valued query to a ReLU network reveals implicit geometric information about activation-region boundaries. The attack exploits this geometric leakage, not statistical bias or reduced-round weakness.

We formalized the key-recovery mechanism via a local separability lemma for natural XOR layers with corner parameter $c < 1$, and validated the attack experimentally over 1000 independent random keys for each AES variant. The success rate was 100% across all tested parameter regimes. Crucially, the vulnerability is independent of key size, round count, or key-schedule diffusion; it arises solely from exposing a natural XOR layer as the first key-dependent transformation in a piecewise-linear network. Conceptually, this work identifies a new attack surface: *linear-region leakage in neural cryptographic realizations*. Unlike differential or linear cryptanalysis, the attack does not analyze cipher structure inside $\mathbb{F}_2$. Instead, it exploits how Boolean operations are embedded into $\mathbb{R}$ via ReLU activations. The leakage stems from the geometry of activation patterns rather than probabilistic imbalance.

More broadly, our findings suggest that correctness-preserving neural implementations of cryptographic primitives are not automatically security-preserving under real-valued oracle access. Any deployment of cryptographic functionality inside continuous architectures must therefore account for linear-region distinguishability at key-dependent layers.

Future directions include:

- (i) Formal security definitions for neural implementations of cryptographic primitives.
- (ii) Verification of the usefulness of the countermeasure against DNN-based attacks in [10]
- (iii) Construction of ReLU-based realizations that eliminate XOR-first separability.

- (iv) Extensions of the analysis to other symmetric primitives and post-quantum cryptography.

In conclusion, while AES remains secure in its classical discrete model, its natural sum-of-corners neural realization admits efficient key recovery under a continuous-query DNN threat model. This separation highlights a fundamental gap between Boolean security and geometric security in neural computation.

Ethical Statement and Responsible Disclosure: The authors recognize the dual-use potential of the cryptanalytic techniques presented. To mitigate risks, we have focused on the theoretical proof of geometric leakage rather than providing a ready-to-use exploitation tool. The raw data and specific attack scripts are withheld from the public domain and will be shared only with verified researchers for defensive purposes.

References

1. Abadi, M., Andersen, D.G.: Learning to protect communications with adversarial neural cryptography. arXiv preprint arXiv:1610.06918 (2016), <https://arxiv.org/abs/1610.06918> 23
2. Bellini, E., Hambitzer, A., Protopapa, M., Rossi, M.: Limitations of the use of neural networks in black box cryptanalysis. In: International Conference on Information Technology and Communications Security. pp. 100–124. Springer (2021) 23
3. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. In: Advances in Cryptology — CRYPTO ’90. pp. 2–21. Lecture Notes in Computer Science, Springer (1991). https://doi.org/10.1007/3-540-38424-3_1 1
4. Carlet, C.: Boolean functions for cryptography and error correcting codes. In: Crama, Y., Hammer, P.L. (eds.) Boolean Models and Methods in Mathematics, Computer Science, and Engineering, pp. 257–397. Springer, New York, NY, USA (2010). https://doi.org/10.1007/978-0-387-93873-0_7 3
5. Courtois, N.T., Shamir, A., Brier, J., Clavier, C.: Algebraic cryptanalysis of the data encryption standard. In: Advances in Cryptology – ASIACRYPT (2007), <https://dl.acm.org/doi/10.5555/1782574.1782588>, extended technical report available 1
6. Coutinho, M., Albuquerque, R., Borges, F., García Villalba, L., Kim, T.H.: Learning perfectly secure cryptography to protect communications with adversarial neural cryptography. *Sensors* **18**(5), 1306 (2018). <https://doi.org/10.3390/s18051306>, <https://doi.org/10.3390/s18051306> 23
7. Cybenko, G.: Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* **2**(4), 303–314 (1989) 22
8. DeepMind: AlphaGo defeats Lee Sedol in historic 4–1 victory. <https://deepmind.com/research/alphago/lee-sedol-2016-match> (2016), accessed: 2026-02-07 1
9. Draguns, A., Gritsevskiy, A., Motwani, S.R., Rogers-Smith, C., Ladish, J., de Witt, C.S.: Unelicitable backdoors in language models via cryptographic transformer circuits. arXiv preprint arXiv:2406.02619 (2024). <https://doi.org/10.48550/arXiv.2406.02619>, <https://arxiv.org/abs/2406.02619> 23
10. Gérault, D., Hambitzer, A., Ronen, E., Shamir, A.: How to securely implement cryptography in deep neural networks. Report 288, IACR Cryptology ePrint Archive (2025), <https://eprint.iacr.org/2025/288.pdf> 2, 6, 11, 24
11. Goldwasser, S., Kim, M.P., Vaikuntanathan, V., Zamir, O.: Planting undetectable backdoors in machine learning models. In: Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS). pp. 931–942. IEEE (2022) 23
12. Hornik, K., Stinchcombe, M., White, H.: Multilayer feedforward networks are universal approximators. *Neural Networks* **2**(5), 359–366 (1989) 22
13. Kalavasis, A., Karbasi, A., Oikonomou, A., Sotiraki, K., Velegkas, G., Zampetakis, M.: Injecting undetectable backdoors in obfuscated neural networks and language models. In: Advances in Neural Information Processing Systems (NeurIPS) (2024), <https://openreview.net/forum?id=KyVBzkCon023>
14. Kanter, I., Kinzel, W., Kanter, E.: Secure exchange of information by synchronization of neural networks. *Europhysics Letters* **57** (2002). <https://doi.org/10.1209/epl/i2002-00552-9> 23
15. Klimov, A., Mityagin, A., Shamir, A.: Analysis of neural cryptography. In: Advances in Cryptology – ASIACRYPT 2002. pp. 288–298. Springer (2002) 23
16. Knudsen, L.R.: Truncated and higher order differentials. In: Fast Software Encryption (FSE 1994). Lecture Notes in Computer Science, vol. 1008, pp. 196–211. Springer (1994). https://doi.org/10.1007/3-540-60590-8_16 1

17. Kocher, P.C.: Timing attacks on implementations of diffie–hellman, RSA, DSS, and other systems. In: Advances in Cryptology – CRYPTO ’96. Lecture Notes in Computer Science, vol. 1109, pp. 104–113. Springer (1996). https://doi.org/10.1007/3-540-68697-5_9 1
18. Lai, X.: Higher order derivatives and differential cryptanalysis. In: Blahut, R. (ed.) Communications and Cryptography, pp. 227–233. Kluwer Academic Publishers (1994) 1
19. Mann, Z.Á., Weinert, C., Chabal, D., Bos, J.W.: Towards practical secure neural network inference: The journey so far and the road ahead. ACM Computing Surveys **56**(5), 1–37 (2023) 24
20. Matsui, M.: Linear cryptanalysis method for DES cipher. In: Advances in Cryptology – EUROCRYPT ’93. Lecture Notes in Computer Science, vol. 765, pp. 386–397. Springer (1994). https://doi.org/10.1007/3-540-48285-7_33 1
21. McCulloch, W.S., Pitts, W.: A logical calculus of the ideas immanent in nervous activity. Bulletin of Mathematical Biophysics **5**, 115–133 (1943) 22
22. Minsky, M., Papert, S.: Perceptrons: An Introduction to Computational Geometry. MIT Press (1969) 22, 23
23. National Bureau of Standards / National Institute of Standards and Technology: Data encryption standard (des), federal information processing standards publication 46. Tech. Rep. FIPS PUB 46, U.S. Department of Commerce, NIST (1977), <https://csrc.nist.gov/pubs/fips/46/final>, withdrawn. Superseded by later revisions (FIPS 46-1, 46-2, 46-3). 1
24. National Institute of Standards and Technology: Advanced encryption standard (aes). FIPS PUB 197 (2001), <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf> 7, 12
25. Rosenblatt, F.: The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review **65**, 386–408 (1958). <https://doi.org/10.1037/h0042519> 22
26. of Standards, N.I., (NIST), T., Dworkin, M.J., Turan, M.S., Mouha, N.: Advanced encryption standard (aes) (May 09 2023). <https://doi.org/10.6028/NIST.FIPS.197-upd1>, https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=936594, updated FIPS 197 AES Standard 2
27. Zhu, B., Chen, K., Lai, X.: Bitwise higher order differential cryptanalysis. In: Trusted Systems – INTRUST 2009. Lecture Notes in Computer Science, vol. 6163, pp. 250–262. Springer (2009). https://doi.org/10.1007/978-3-642-14597-1_16 1